

Production Agent Reliability & Security Platform

A deployable product specification for trustworthy RAG and tool-using agents

Core product idea: Build an independent control plane that allows teams to deploy AI agents safely by enforcing policy before tool execution, tracing every step, evaluating complete trajectories, detecting behavioral anomalies, and providing rollback or human approval when risk exceeds a defined threshold.

Product name candidate: AgentShield Control Plane. Do not present it as an NVIDIA product or MIT project. Use NVIDIA and MIT materials as public technical references and design inspiration.

1. Why Production Agents Fail

Failure class	Real production problem	Product response
Quality and grounding	Agents produce plausible but wrong answers, retrieve incomplete evidence, or succeed only on curated demos.	Golden-task registry, adversarial test sets, trajectory evaluation, claim/citation checks, abstention.
Non-determinism	The same request can choose different plans, tools, or outputs, making debugging difficult.	Immutable run IDs, prompt/model/tool versions, replayable traces, deterministic policies, decision snapshots.
Tool misuse and excessive agency	A model can call the wrong API, use excessive permissions, or perform an irreversible action.	Capability registry, least privilege, risk tiers, policy engine, dry-run, human approval, postcondition checks.
Prompt injection and tool poisoning	Instructions hidden in documents, web pages, tool descriptions, or outputs can redirect the agent.	Untrusted-content labels, instruction quarantine, tool metadata scanning, signed/versioned tools, runtime isolation.
Data leakage and privacy	Sensitive data may enter prompts, logs, retrieved context, tool arguments, or external destinations.	Tenant isolation, PII/secret redaction, dataflow policies, destination controls, encrypted audit logs.
Silent failures	A workflow reports success although a tool call failed, a record was partially changed, or the answer is unsupported.	State assertions, postcondition verification, outcome validators, anomaly alerts, human escalation.
Cost and runaway loops	Agents repeat calls, consume excessive tokens, or trigger expensive workflows.	Budgets, loop limits, token/cost telemetry, circuit breakers, model routing, per-tenant quotas.
Operations and scale	Prototype code cannot survive concurrent users, model outages, version changes, or deployment drift.	Queue-based execution, retries with idempotency, circuit breakers, health checks, horizontal scaling, canary releases.
Governance and accountability	Teams cannot prove why an action occurred or who approved it.	Tamper-evident audit trail, policy decisions, approvals, provenance, retention controls, exportable incident reports.

These failure classes align with production discussions emphasizing quality, observability, evaluation, governance, security, and economics as separate readiness dimensions. [web:49][web:50]

2. Reference Landscape and Design Lessons

Reference	What it provides	What your product should learn
NVIDIA Agentic AI / Agent Toolkit	Open models, verified skills, runtime controls, evaluation, guardrails, observability, and deployment building blocks.	Design a modular control plane rather than a single chatbot; separate model, tools, policy, runtime, and evaluation layers. [web:46]
NVIDIA NeMo	NeMo Evaluator, Retriever, Guardrails, Agent Toolkit, NIM, NIM Operator, and lifecycle tooling.	Support evaluation-based development, continuous improvement, production deployment, and framework-agnostic traces. [web:67]
NVIDIA NeMo Guardrails	Input/output rails can protect the model path, but NVIDIA documentation explicitly notes they do not automatically cover tool misuse or instructions injected through tool output.	Make tool-call authorization and tool-output inspection first-class features, not optional prompt filters. [web:61]

NVIDIA NIM	Containerized inference, stable production branches, structured logs, Prometheus metrics, and distributed tracing.	Use model-provider abstraction plus health checks, metrics, tracing, and versioned deployments. [web:63][web:68]
NVIDIA Enterprise RAG guide	Production deployment patterns involving ingestion, vector databases, NIM services, scaling, and Kubernetes-oriented infrastructure.	Design for real deployment: queues, indexes, scaling, health, secrets, and repeatable infrastructure—not only a notebook. [web:69]
OWASP LLM risks	Prompt injection, information disclosure, supply chain, poisoning, improper output handling, excessive agency, leakage, vector weaknesses, misinformation, and unbounded consumption.	Map every product control to a threat and test it with attack cases. [web:81]
NIST AI RMF / GenAI Profile	Govern, map, measure, and manage AI risks across the lifecycle.	Maintain a risk register, intended-use boundary, evaluation evidence, incident process, and change management. [web:87]
OpenTelemetry GenAI conventions	A standard direction for GenAI telemetry.	Emit trace/span attributes for model calls, retrieval, tool calls, token usage, errors, and policy decisions. [web:65]

3. Full Product Problem Statement

Organizations are connecting LLM agents to private knowledge, code repositories, ticketing systems, databases, cloud resources, and communication tools. Existing prototypes can demonstrate impressive task completion, but they do not reliably answer five operational questions: What did the agent do? Why did it do it? Was the action authorized? Did the outcome actually satisfy the task? Can the team stop, replay, investigate, and improve it without exposing sensitive data?

The proposed system solves this by providing a deployable, model-agnostic reliability and security control plane for RAG and tool-using agents. It intercepts model, retrieval, and tool interactions; classifies data and actions by trust and risk; enforces least-privilege policy; requires meaningful approval for consequential operations; evaluates complete execution trajectories; verifies postconditions; records privacy-preserving telemetry; and continuously detects quality, safety, cost, and behavioral regressions.

Research problem: How can an agent control plane reduce unsafe or unsupported agent actions while preserving useful task completion, under realistic multi-step workflows, adversarial retrieved content, model variation, tool failures, and production cost limits?

Engineering objective: Deliver a self-hostable platform that a small company can run locally or in the cloud, connect to an existing agent through an OpenAI-compatible proxy or SDK, register tools with explicit permissions, observe runs through a dashboard, execute an evaluation suite, and safely promote a workflow from sandbox to production.

4. Proposed Architecture

Client/Agent SDK → API gateway → identity and tenant isolation → policy decision point → model gateway → retrieval gateway → tool registry → sandboxed tool runner → approval service → postcondition verifier → event bus → OpenTelemetry collector → metrics/logs/traces → evaluation and incident dashboard.

- Control plane: projects, tenants, environments, policies, approvals, budgets, model/tool versions, and deployment promotion.
- Data plane: low-latency proxy and execution path that evaluates requests without exposing secrets to the model.
- Tool registry: schemas, owner, version, permissions, data sensitivity, network destinations, signature/hash, and change history.
- Policy engine: declarative rules for identity, task, tool, arguments, dataflow, destination, risk, budget, and approval.
- Runtime: isolated containers or sandboxes with filesystem, network, credential, CPU, memory, and time limits.
- Evaluation engine: golden tasks, attack cases, trajectory graders, citation checks, regression comparisons, and release gates.
- Observability: step-level traces, model/retrieval/tool spans, policy decisions, token/cost data, errors, outcomes, and privacy-safe payload references.

5. Product MVP That Is Actually Deployable

- OpenAI-compatible gateway supporting at least two model providers or one local model plus one API model.
- RAG connector with document trust labels, metadata, versioning, hybrid retrieval, and evidence references.
- Tool registry with read/write/destructive risk classes and explicit scopes.
- Policy engine with allow, deny, require approval, redact, and sandbox decisions.
- Sandbox runner for search, file read, database read, ticket creation, and simulated email—not real destructive integrations at first.
- Run dashboard showing every model call, retrieved item, tool argument, policy result, approval, output, latency, tokens, cost, and final status.

- Evaluation CLI that runs a golden dataset and adversarial suite before deployment.
- Incident controls: kill switch, revoke tool, revoke credential, quarantine workflow, replay run, and export report.
- Docker Compose deployment first; Kubernetes manifests only after the single-node product is stable.

6. Non-Negotiable Evaluation Targets

Dimension	Initial target
Safety	100% of simulated destructive actions require approval; zero secrets intentionally exposed in test cases.
Security	Demonstrate reduced attack success against direct/indirect prompt injection and tool-poisoning test suites; publish false-positive trade-offs.
Grounding	At least 95% citation coverage on answerable benchmark claims, with unsupported claims reported rather than hidden.
Reliability	At least 95% successful completion on defined benign workflows with tool failures injected; no duplicate side effects under retry.
Observability	Every run has a correlated trace, policy decision, tool outcome, and cost/latency record.
Economics	Per-task budget enforcement with graceful stop or model downgrade when limits are reached.
Reproducibility	A new user can run the demo and evaluation locally from documented commands.

Targets are proposed engineering goals, not claims about industry standards. Report the measured baseline, confidence intervals where possible, and failures.

7. First Real Customer Wedge

Do not start with “an agent for everything.” Choose one high-value but bounded workflow. The strongest wedge for your background is a **secure engineering support and incident-triage agent** that reads approved documentation, logs, tickets, and runbooks; proposes diagnosis with citations; executes only reversible diagnostic tools automatically; and requires approval for changes. This connects your RAG, cybersecurity, systems, and startup interests while allowing a real user to test it.

Example workflow: ingest service documentation and runbooks → receive an incident ticket → retrieve relevant evidence → inspect approved logs → propose ranked causes → ask for missing information → create a draft remediation plan → execute a read-only check → require approval for a change → verify the postcondition → produce an auditable incident report.

8. Honest Positioning

Use this wording: “Independent, self-hostable control plane for production RAG and tool-using agents, inspired by public NVIDIA NeMo/NIM architecture patterns, OpenTelemetry GenAI conventions, OWASP LLM risks, and NIST AI risk-management practices.”

Do not claim NVIDIA partnership, MIT affiliation, official certification, or compatibility with NVIDIA products unless you implement and test that compatibility. You can later add optional NVIDIA integrations such as NIM endpoints, NeMo Guardrails, NeMo Evaluator, or NeMo Agent Toolkit adapters. NVIDIA documentation itself distinguishes model-path guardrails from tool misuse and tool-output injection, which makes your broader control-plane scope technically defensible. [web:61]

9. Build Sequence

- Phase 1: Build the proxy, event schema, trace viewer, tool registry, and simulated tools.
- Phase 2: Add policy decisions, approval workflow, budgets, sandboxing, and postcondition checks.
- Phase 3: Add RAG trust labels, citations, hybrid retrieval, claim validation, and evaluation datasets.
- Phase 4: Add attack suite for prompt injection, tool poisoning, data leakage, excessive agency, and runaway loops.
- Phase 5: Add deployment packaging, multi-user authentication, tenant isolation, secret management, health checks, and failure recovery.
- Phase 6: Pilot with one real engineering workflow, collect failure traces, and publish a measured technical report.

This project becomes “real” when another person can deploy it, connect a bounded workflow, observe an agent run, reproduce an unsafe case, see the policy stop it, approve a safe action, and inspect evidence that the final state is correct.